# Goals

- Provide a drop-in npm package that customers can embed on their website/app.
- The embedded UI supports two experiences:
  - Builder mode: a conversational assistant that turns instructions into an Activepieces flow.
  - Agent mode: a conversational assistant that triggers an existing client flow based on chat messages and returns the result.

# What the repo already provides (building blocks)

- Flow creation is already a service call in the API:

```
export const flowService = (log: FastifyBaseLogger) => ({
    async create({ projectId, request, externalId, ownerId, templateId, creatorPlatform
        ...
        const savedFlow = await flowRepo().save(newFlow)
        const savedFlowVersion = await flowVersionService(log).createEmptyVersion(
            savedFlow.id,
            {
                displayName: request.displayName,
                notes: [],
            },
        )
        ...
    },
```

- The codebase already exposes an “agent-friendly” tool surface (MCP) for programmatic flow building:
  - Set/update trigger: `ap_update_trigger` .

```
export const apUpdateTriggerTool = (mcp: McpServer, log: FastifyBaseLogger): McpToolDef
  return {
    title: 'ap_update_trigger',
    ...
    execute: async (args) => {
      ...
      const operation: FlowOperationRequest = {
        type: FlowOperationType.UPDATE_TRIGGER,
```

- Add steps: `ap_add_step` .

```
export const apAddStepTool = (mcp: McpServer, log: FastifyBaseLogger): McpToolDefinitio
  return {
    title: 'ap_add_step',
    description: 'Add a new step to a flow (skeleton only — configure with ap_update_st
    ...
    execute: async (args) => {
      ...
      const operation: FlowOperationRequest = {
        type: FlowOperationType.ADD_ACTION,
```

- The MCP server includes a documented recommended workflow (create → trigger → steps → validate → publish).

```
const MCP_SERVER_INSTRUCTIONS = `## Activepieces MCP Server — Agent Workflow Guide

### Recommended workflow
1. **Discover**: ap_list_pieces ...
2. **Schema**: ap_get_piece_props ...
3. **Build**: ap_create_flow → ap_update_trigger → ap_add_step → ap_update_step
4. **Validate**: ap_validate_step_config ... or ap_validate_flow ...
5. **Publish**: ap_lock_and_publish → ap_change_flow_status
```

- Flow triggering from outside already exists via a webhook endpoint ( `/:flowId` and `/:flowId/sync` ):

```
app.all(
  '/:flowId/sync',
  ...,
  async (request, reply) => {
    const response = await webhookService.handleWebhook({
      ...,
      flowId: request.params.flowId,
      async: false,
      ...
    })
```

- There is already an embeddable UI SDK for Activepieces screens (including the flow builder).

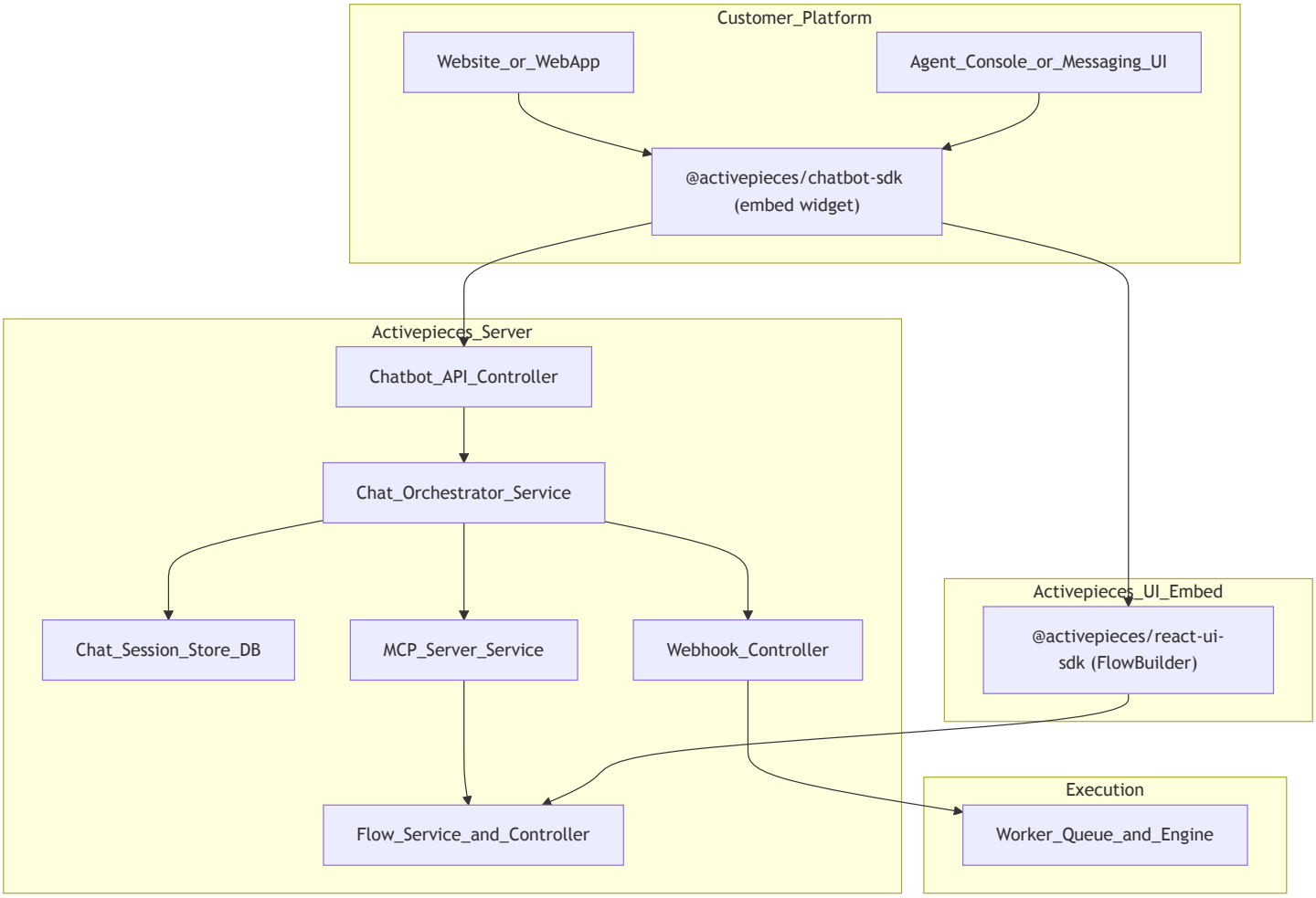
```
export const FlowBuilder: React.FC<FlowBuilderProps> = (props) => {
  const { apiUrl, token, projectId, flowId } = props;
  useEffect(() => {
    configureAPI({ apiUrl, token, projectId, flowId });
  }, [apiUrl, token, projectId, flowId]);
  ...
}
```

# Proposed architecture

- `@activepieces/chatbot-sdk` (new package under `packages/extensions/` ):
  - `ChatbotWidget` (React) + optional vanilla JS `mountChatbot()` helper (mirrors `react-ui-sdk` pattern).
  - Optional wrappers for Angular (similar pattern to `react-ui-sdk` , if required).
- A new server capability for “chat sessions” (minimal):
  - Persist chat session state (conversation history + selected flow + tool calls) scoped to a project.
  - Expose a single API endpoint the widget can call: `POST /v1/chatbot/message` .

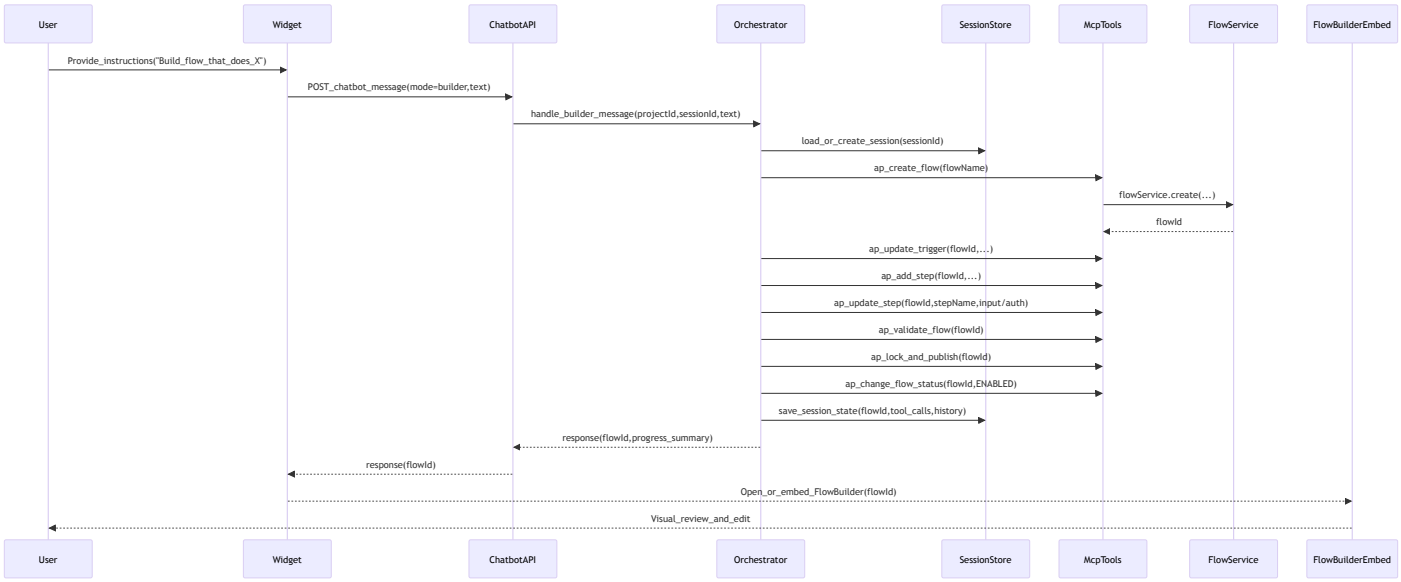
# Diagrams

## Architecture (high level)

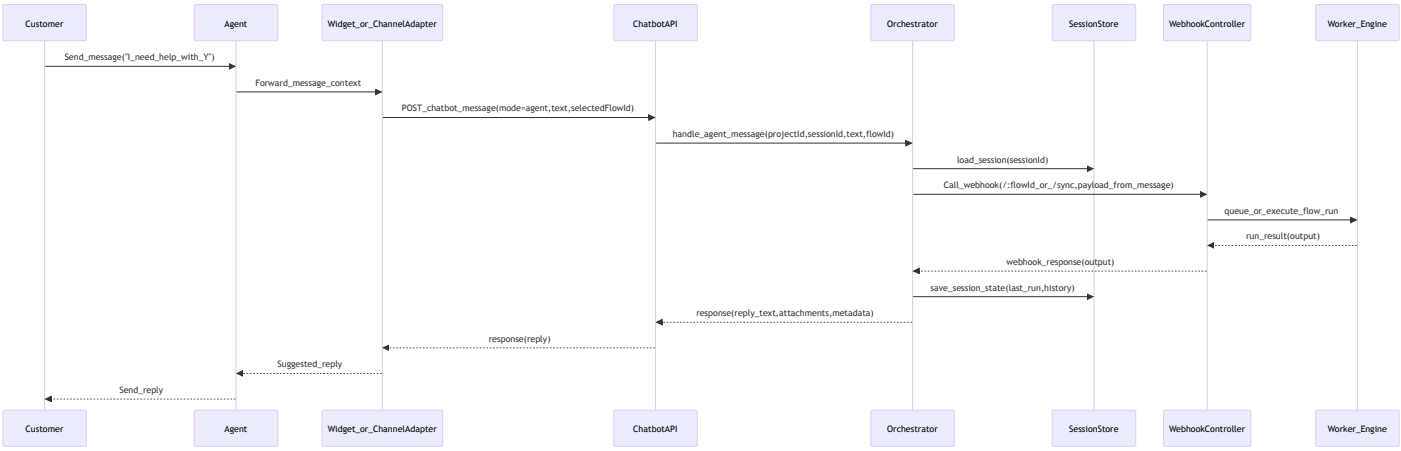



Note: The diagram uses a top-to-bottom ( `TB` ) outer layout and stacks `Customer_Platform` → `Activepieces_UI_Embed` → `Activepieces_Server` → `Execution` so Mermaid does not overlap the embed subgraph with the server subgraph (a common issue with `LR` + multiple wide subgraphs). Node labels that include `@` or `/` (npm scopes) must use **quoted** text, e.g. `Widget["@activepieces/..."]` , or Markdown Preview Enhanced / Mermaid will throw a parse error.

# Sequence diagram 1: Builder mode (instruction → auto-build flow)




# Sequence diagram 2: Agent mode (message → trigger existing flow → reply)




# Builder mode (create/update flow from instructions)

- Widget sends user instruction text.
- Server-side “chat orchestrator” uses MCP tool workflow:
  - `ap_create_flow` .
  - `ap_update_trigger` .
  - `ap_add_step` / `ap_update_step` .
  - `ap_validate_flow` .
  - `ap_lock_and_publish` and enable flow.

- Widget deep-links (or embeds) `FlowBuilder` from `@activepieces/react-ui-sdk` so the user can visually review the generated flow.

### Agent mode (trigger an existing flow)

- Widget (or channel adapter) maps an inbound customer message into a run request.
- Server triggers the flow by calling the existing webhook endpoint ( `/:flowId` or `/:flowId/sync` ) and returns the result back to the widget/channel.

## Security + tenancy model (decision points, but implementable incrementally)

- Start with "Activepieces-hosted AI orchestration" where the widget authenticates with a project-scoped token.
- Later support "customer BFF" mode where the widget never talks to Activepieces directly; customer backend calls Activepieces.

## Implementation steps

1. Server: add a new module for chatbot orchestration (controller + service) that:
   - Accepts `projectId` + message + `mode` .
   - In builder mode, drives the MCP tool chain for flow creation and returns `{flowId}` plus human-readable progress.
   - In agent mode, triggers a configured flow via the webhook controller path and returns the response.
2. Server: add minimal persistence for chat sessions (DB entity + repo) so multi-turn conversations work.
3. SDK: create a new extension package with:
   - React `ChatbotWidget` component.
   - A `mountChatbot()` helper for plain HTML sites.
   - Styling aligned with existing embed patterns.
4. SDK: integrate optional embedded `FlowBuilder` (from `@activepieces/react-ui-sdk` ) for "review/edit generated flow".
5. Verification: add a small integration test that:
   - Creates a flow via the chatbot orchestration.
   - Triggers it via webhook and asserts expected output.

# MVP scope recommendation

- Limit initial supported “instruction types” to a small set of patterns (e.g., “when I receive a message → call webhook/LLM → send reply”), then expand piece coverage after the end-to-end path is stable.